

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc Lập – Tự Do – Hạnh Phúc

ĐỀ CƯƠNG CHI TIẾT
ĐỒ ÁN CHUYÊN NGÀNH

HỌC KỲ 2 – NĂM HỌC 2025–2026

TÊN ĐỀ TÀI TIẾNG VIỆT:	Thiết kế và Triển khai Quy trình DevSecOps Kết hợp GitOps cho Hệ thống Microservices trên Nền tảng AWS
TÊN ĐỀ TÀI TIẾNG ANH:	Design and Implementation of a DevSecOps Pipeline Combined with GitOps for a Microservices System on AWS
Cán bộ hướng dẫn:	ThS. Lê Anh Tuấn
Thời gian thực hiện:	Từ ngày 17/02/2026 đến ngày 27/06/2026
Sinh viên thực hiện:	Hồ Nhật Minh – 23520924 – MMTT2023

Tổng quan đề tài

Trong bối cảnh chuyển đổi số, các tổ chức đang chuyển từ kiến trúc monolithic truyền thống sang kiến trúc microservices để đạt được khả năng mở rộng và tốc độ phát triển nhanh hơn. Tuy nhiên, việc triển khai và vận hành hàng chục microservice đặt ra những thách thức lớn về bảo mật, nhất quán và khả năng kiểm soát. Các quy trình triển khai thủ công truyền thống (manual deployment, manual security review) không còn phù hợp, dẫn đến nguy cơ cấu hình sai (misconfiguration), lỗ hổng bảo mật không được phát hiện kịp thời, và khó khăn trong rollback.

Hai phương pháp luận hiện đại đang được cộng đồng DevOps áp dụng rộng rãi để giải quyết bài toán này:

- DevSecOps** – tích hợp bảo mật vào mọi giai đoạn của vòng đời phát triển phần mềm (shift-left security), bao gồm SAST (Static Application Security Testing), SCA (Software Composition Analysis), và container image scanning. Thay vì kiểm tra bảo mật ở bước cuối cùng, DevSecOps đưa các “security gate” vào pipeline CI/CD.
- GitOps** – sử dụng Git repository làm nguồn sự thật duy nhất (single source of truth) cho trạng thái hệ thống. Một operator như Argo CD liên tục so sánh trạng thái thực tế trên Kubernetes cluster với trạng thái khai báo trong Git, và tự động đồng bộ khi có sự khác biệt. Điều này giúp audit trail hoàn chỉnh và rollback dễ dàng (chỉ cần git revert).

Tuy nhiên, việc kết hợp DevSecOps và GitOps thành một quy trình thống nhất cho microservices trên AWS vẫn là một bài toán chưa có nhiều tài liệu hướng dẫn thực tiễn, đặc biệt trong môi trường học thuật tại Việt Nam. Các công cụ như SonarQube, OWASP Dependency Check, Trivy,

Jenkins, Argo CD, và Terraform thường được sử dụng riêng lẻ, nhưng việc tích hợp chúng thành một pipeline end-to-end đòi hỏi kiến thức liên ngành (DevOps, Security, Cloud, Kubernetes).

Trong phạm vi đề án này, sinh viên sẽ xây dựng một hệ thống minh họa toàn bộ quy trình DevSecOps + GitOps cho một ứng dụng microservices mẫu (retail store) trên AWS, sử dụng Terraform để quản lý hạ tầng (Infrastructure as Code) và Jenkins/Argo CD để tự động hóa CI/CD. Hệ thống sẽ chứng minh được rằng mọi thay đổi mã nguồn đều trải qua các bước kiểm thử bảo mật trước khi đến production, và mọi thay đổi production đều có audit trail trong Git.

Mục tiêu của đề tài

Mục tiêu tổng quát của đề án là thiết kế và triển khai thành công một quy trình DevSecOps kết hợp GitOps, hoạt động được trên AWS với một ứng dụng microservices thực tế. Các mục tiêu cụ thể bao gồm:

2.1 Tự động hóa hạ tầng bằng Infrastructure as Code (IaC)

Toàn bộ hạ tầng AWS (VPC, EKS cluster, IAM roles, ECR repositories) được định nghĩa và quản lý bằng Terraform. Terraform state được lưu tập trung trên S3 với cơ chế lock qua DynamoDB, đảm bảo tính nhất quán và khả năng tái tạo (reproducibility).

Cải tiến so với thực trạng: Hiện nay nhiều sinh viên và doanh nghiệp nhỏ vẫn tạo tài nguyên AWS thủ công qua Console, dẫn đến khó kiểm soát phiên bản và khó tái tạo môi trường. Sử dụng IaC giúp mọi thay đổi hạ tầng được version control và review như code.

2.2 Tích hợp bảo mật vào CI pipeline (Shift-Left Security)

Pipeline CI (Jenkins) sẽ tự động thực hiện các bước kiểm thử bảo mật trước khi cho phép deploy:

- **SAST** (SonarQube): phân tích mã nguồn tĩnh để phát hiện lỗ hổng.
- **SCA** (OWASP Dependency Check): kiểm tra lỗ hổng trong thư viện bên thứ ba.
- **Container Scan** (Trivy): quét Docker image trước khi push lên ECR.

Cải tiến so với thực trạng: Quy trình truyền thống thường kiểm tra bảo mật thủ công và muộn, dẫn đến lỗ hổng chỉ được phát hiện sau khi đã deploy. Shift-left giúp phát hiện sớm ngay trong giai đoạn phát triển.

2.3 Triển khai GitOps với Argo CD

Sử dụng mô hình 2-repository:

- **App Repo:** chứa mã nguồn microservices, kích hoạt CI pipeline khi có push.
- **GitOps Repo:** chứa manifest triển khai (Kustomize). CI pipeline tự động cập nhật image tag. Argo CD giám sát repo này và tự đồng bộ lên EKS.

Cải tiến so với thực trạng: Triển khai truyền thống dùng kubectl apply thủ công không có audit trail và khó rollback. GitOps cho phép rollback = git revert, và toàn bộ lịch sử triển khai được lưu trong Git commit history.

2.4 Giám sát tập trung với CloudWatch + Grafana

Thiết lập dashboard giám sát trạng thái cluster và ứng dụng, cấu hình alerts cho các tình huống bất thường (node down, CPU cao, pod crash).

2.5 Chứng minh tính khả thi trên môi trường thực

Triển khai thành công toàn bộ 5 microservices của ứng dụng bán lẻ mẫu (aws-containers/retail-store-sample-app) lên EKS thông qua pipeline DevSecOps + GitOps tự động, và thực hiện demo rollback bằng git revert.

Phương pháp thực hiện

3.1 Nghiên cứu và khảo sát

- Nghiên cứu lý thuyết về DevSecOps, GitOps, Kubernetes, và AWS EKS.
- Khảo sát các công cụ: Terraform, Jenkins, SonarQube, OWASP DC, Trivy, Argo CD, Kustomize, Helm, Prometheus, Grafana.
- Phân tích kiến trúc microservices của sample app (5 services: UI, Cart, Orders, Catalog, Checkout).

3.2 Thiết kế kiến trúc hệ thống

- Vẽ sơ đồ kiến trúc tổng thể (Mermaid) thể hiện luồng dữ liệu từ Developer đến Production.
- Thiết kế cấu trúc workspace gồm 10 danh mục: admin, docs, repos, infra, platform, ci, security, observability, evidence, scripts.
- Lập kế hoạch triển khai theo 4 phase.

3.3 Triển khai hạ tầng (Infrastructure as Code)

- Sử dụng Terraform để provision: S3 + DynamoDB backend, VPC (10.0.0.0/16), EKS cluster v1.33, managed node groups (t3.medium), ECR repositories (5 repos).
- Sử dụng module terraform-aws-modules/vpc và terraform-aws-modules/eks.
- Cài đặt Kubernetes addons: VPC-CNI, CoreDNS, kube-proxy, AWS Load Balancer Controller, Cert Manager.

3.4 Xây dựng CI Pipeline

- Viết Jenkinsfile với các stage: Checkout, Build, SonarQube SAST, OWASP Dependency Check, Docker Build, Trivy Image Scan, Push ECR, Update GitOps Repo.
- Tích hợp webhook từ GitHub để tự động trigger pipeline.
- Cấu hình Jenkins agent chạy trên EC2 hoặc Docker container.

3.5 Xây dựng CD Pipeline (GitOps)

- Viết Kustomize manifests: base (chung) + overlays (dev, staging, prod).
- Cài đặt Argo CD lên EKS, cấu hình Application resource trỏ đến GitOps repo.
- Tự động sync: Argo CD phát hiện thay đổi trong GitOps repo và đồng bộ lên cluster.

3.6 Kiểm thử và đánh giá

- Kiểm tra toàn bộ pipeline hoạt động end-to-end: push code → Jenkins build + scan → push ECR → Argo CD sync → EKS.

- Thực hiện demo rollback: git revert commit trong GitOps repo → Argo CD tự động quay về phiên bản cũ.
- Đánh giá hiệu quả của security gates: kiểm tra xem pipeline có chặn được code có lỗi hổng không.
- Đánh giá thời gian end-to-end từ code push đến deploy.

Các nội dung chính và giới hạn của đề tài

4.1 Nội dung chính

1. **Hạ tầng AWS quản lý bằng Terraform:** S3 + DynamoDB backend, VPC, EKS cluster v1.33 với managed node groups, ECR repositories cho 5 microservices, IAM roles và security groups.
2. **CI Pipeline (Jenkins):** Tự động build, SAST (SonarQube), SCA (OWASP Dependency Check), Docker build, container scan (Trivy), push ECR, cập nhật GitOps repo.
3. **CD GitOps (Argo CD):** Triển khai Kustomize-based manifests, Argo CD tự động sync từ GitOps repo lên EKS.
4. **Security Gates:** 3 checkpoint bảo mật trong CI pipeline, đảm bảo chỉ code và image “sạch” mới được deploy.
5. **Observability:** CloudWatch log groups, Prometheus + Grafana dashboards cho EKS cluster và ứng dụng.
6. **Demo rollback:** Minh họa khả năng rollback nhanh bằng git revert trên GitOps repo.
7. **Tài liệu đầy đủ:** Báo cáo tổng kết, sơ đồ kiến trúc, hướng dẫn triển khai, bằng chứng (evidence) cho từng phase.

4.2 Giới hạn

1. **Quy mô:** Hệ thống triển khai trên một EKS cluster với 2 managed node groups, không triển khai multi-cluster hoặc multi-region.
2. **Môi trường:** Chỉ triển khai môi trường dev. Môi trường staging và prod được chuẩn bị cấu trúc (Kustomize overlays) nhưng không triển khai đầy đủ.
3. **CI/CD:** Sử dụng Jenkins cho CI và Argo CD cho CD, không triển khai các công cụ thay thế như GitHub Actions, GitLab CI, hay Flux CD.
4. **Security Scanning:** Chỉ dùng SAST (SonarQube), SCA (OWASP DC), và container scan (Trivy). Không bao gồm DAST, penetration testing, hay secret scanning.
5. **Auto-scaling:** Cluster sử dụng node group có min/max cố định, không triển khai Karpenter hoặc Cluster Autoscaler.
6. **Service Mesh:** Không triển khai Istio hoặc Linkerd. Giao tiếp giữa các service qua Kubernetes Service cơ bản.
7. **HTTPS/TLS:** Cert Manager được cài đặt nhưng chưa cấu hình Ingress với TLS do cần domain thật và DNS.
8. **Monitoring:** Dashboard Prometheus + Grafana cài đặt ở mức cơ bản, chưa có alerting rules phức tạp.

4.3 Phương pháp đánh giá hệ thống

- **Demo end-to-end:** Code push → Jenkins build → scan → push ECR → Argo CD sync → pods chạy trên EKS. Tất cả 5 service hoạt động.
- **Demo rollback:** Git revert commit trong GitOps repo → Argo CD tự động quay về phiên bản cũ → kiểm tra pod chạy phiên bản cũ.

- **Demo security gate:** Push code có lỗ hổng (ví dụ dependency có CVE) → pipeline bị chặn ở bước OWASP/Trivy → không deploy.
- **Đo lường:** Thời gian end-to-end pipeline, tỉ lệ phát hiện lỗ hổng của các công cụ scan.
- **Tài liệu evidence:** Chụp màn hình từng bước, lưu trong thư mục 08-evidence.

Kế hoạch thực hiện

5.1 Giai đoạn 1: Lập kế hoạch và thiết lập (Tháng 2 – tháng 3)

- Phân tích đề tài, xác định phạm vi, chọn công nghệ.
- Clone sample app, tạo workspace, vẽ sơ đồ kiến trúc.
- Cài đặt toolchain (Terraform, kubectl, Helm, Docker, AWS CLI).
- Tạo IAM user, cấu hình AWS credentials.
- Viết tài liệu roadmap và hướng dẫn thực hiện.

5.2 Giai đoạn 2: Hạ tầng (Infrastructure as Code) (Tháng 3 – tháng 4)

- Terraform backend: S3 + DynamoDB.
- Provision VPC, EKS cluster, ECR repositories.
- Cài đặt Kubernetes addons (LB Controller, Cert Manager).
- Kiểm thử cluster (kubectl get nodes, pods).

5.3 Giai đoạn 3: CI + Security Pipeline (Tháng 4 – tháng 5)

- Cài đặt và cấu hình Jenkins.
- Viết Jenkinsfile với các stage: build, SAST, SCA, Docker, Trivy, push ECR, update GitOps repo.
- Tích hợp SonarQube, OWASP Dependency Check, Trivy.
- Cấu hình webhook GitHub.

5.4 Giai đoạn 4: CD + Monitoring (Tháng 5 – tháng 6)

- Viết Kustomize manifests (base + dev overlay).
- Cài đặt Argo CD, cấu hình sync tự động.
- Demo deploy toàn bộ 5 service lên EKS.
- Demo rollback bằng git revert.
- Cài đặt CloudWatch + Prometheus + Grafana.
- Viết runbooks, cấu hình alerts.

5.5 Giai đoạn 5: Hoàn thiện và bảo vệ (Tháng 6)

- Tổng hợp evidence (screenshots, logs).
- Viết báo cáo cuối kỳ, slides bảo vệ.
- Dọn dẹp tài nguyên AWS.

Xác nhận của CBHD
(Ký tên và ghi rõ họ tên)

TP. HCM, ngày ... tháng ... năm 2026
Sinh viên thực hiện
(Ký tên và ghi rõ họ tên)

ThS. Lê Anh Tuấn

Hồ Nhật Minh